Assignment No. : 05

Q1. Docker Swarm: Nginx Service

Deploy an Nginx service in Docker Swarm.

Requirements:

1. Initialize Docker Swarm.

```
student@student-VMware-Virtual-Platform:~/website$ docker swarm init
Swarm initialized: current node (7lv0n0s5c579aph00tbtbl12y) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-5cian9gg7eerktnkqnvevjn648pttm81vyd6zudhee6y55nlnz-1s7ylv86rr1a2q61e3ty5vkgq 192.
168.74.117:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.
```

2. Create a service:

   Service name: nginx-service

   Image: nginx

   Replicas: 2

3. Expose container port 80 on host port 8080 .

4. Verify the service using:

   docker service ls

   docker service ps nginx-service

```
student@student-VMware-Virtual-Platform:~/website$ docker  service  create  --name  nginx-service  --replicas  2
 -p 8081:80  newdelthis/custom-nginx:1.0
vr4kga2igl3l985f9b97w1vn1
overall progress: 2 out of 2 tasks
1/2: running   [==================================================>]
2/2: running   [==================================================>]
verify: Service vr4kga2igl3l985f9b97w1vn1 converged
student@student-VMware-Virtual-Platform:~/website$ docker service ls
ID             NAME           MODE         REPLICAS   IMAGE                          PORTS
vr4kga2igl3l   nginx-service  replicated   2/2        newdelthis/custom-nginx:1.0    *:8081->80/tcp
```

5. Access Nginx from the browser.

```
student@student-VMware-Virtual-Platform:~/website$ curl "localhost:8081"
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
```

Q2. Docker Swarm: Python Worker Service

Deploy a Python script as a Docker Swarm service.

Requirements:

1. Create app.py that prints:

Running in Swarm...

```
 GNU nano 8.7.1                                          app.py
import time
import sys
while True:
        print("Running in Swarm...",flush=True)
        time.sleep(5)
```

.

every 5 seconds.

1. Create a Dockerfile.

```
 GNU nano 8.7.1
FROM python:3.11-slim
WORKDIR /app
COPY app.py .
CMD ["python","app.py"]
```

2. Build a Docker image.

```
student@student-VMware-Virtual-Platform:~/my-py-app$ docker build -t my-app:1.0 .
[+] Building 1.8s (9/9) FINISHED                                                docker:default
 => [internal] load build definition from Dockerfile                                    0.0s
 => => transferring dockerfile: 110B                                                    0.0s
 => [internal] load metadata for docker.io/library/python:3.11-slim                     1.4s
 => [auth] library/python:pull token for registry-1.docker.io                           0.0s
 => [internal] load .dockerignore                                                       0.0s
 => => transferring context: 2B                                                         0.0s
 => [1/3] FROM docker.io/library/python:3.11-slim@sha256:673719e071ecf27434fd2635bbd8d  0.0s
 => => resolve docker.io/library/python:3.11-slim@sha256:673719e071ecf27434fd2635bbd8d  0.0s
```

3. Deploy the service:

Service name: python-service Replicas: 3

```
student@student-VMware-Virtual-Platform:~/my-py-app$ docker service create  --name python-service  --replicas 3  my-a
pp:1.0
image my-app:1.0 could not be accessed on a registry to record
its digest. Each node will access my-app:1.0 independently,
possibly leading to different nodes running different
versions of the image.

zr0q3fhexy0th5vdl7wx6ee44
overall progress: 3 out of 3 tasks
1/3: running   [==================================================>]
2/3: running   [==================================================>]
3/3: running   [==================================================>]
verify: Service zr0q3fhexy0th5vdl7wx6ee44 converged
```

4. Check logs from different running containers.

5. Verify task distribution using: docker service ps python-service

```
student@student-VMware-Virtual-Platform:~/my-py-app$ docker service ps python-service
ID             NAME              IMAGE        NODE                              DESIRED STATE   CURRENT STATE
  ERROR     PORTS
6imztwukq4ey   python-service.1   my-app:1.0   student-VMware-Virtual-Platform   Running         Running 42 seconds ago

mriom6cs7s7e   python-service.2   my-app:1.0   student-VMware-Virtual-Platform   Running         Running 41 seconds ago

onnf896xgfyh   python-service.3   my-app:1.0   student-VMware-Virtual-Platform   Running         Running 41 seconds ago

student@student-VMware-Virtual-Platform:~/my-py-app$ docker service logs python-service
python-service.2.mriom6cs7s7e@student-VMware-Virtual-Platform    | Running in Swarm...
python-service.2.mriom6cs7s7e@student-VMware-Virtual-Platform    | Running in Swarm...
python-service.1.6imztwukq4ey@student-VMware-Virtual-Platform    | Running in Swarm...
python-service.1.6imztwukq4ey@student-VMware-Virtual-Platform    | Running in Swarm...
python-service.1.6imztwukq4ey@student-VMware-Virtual-Platform    | Running in Swarm...
```

Q3. Docker Swarm: Scaling Service

Scale an existing Docker Swarm service.

Requirements:

1. Deploy an Nginx service with 2 replicas.

```
student@student-VMware-Virtual-Platform:~/website$ docker service create --name nginx-swarm --replicas 2 -p 8888:80 ngin
x:latest
rxy9xb78d1pdistwn8tuztdp8
overall progress: 2 out of 2 tasks
1/2: running   [==================================================>]
2/2: running   [==================================================>]
verify: Service rxy9xb78d1pdistwn8tuztdp8 converged
student@student-VMware-Virtual-Platform:~/website$ docker service ps nginx-swarm
ID             NAME            IMAGE          NODE                              DESIRED STATE   CURRENT STATE
 ERROR     PORTS
udjccfahrr3l   nginx-swarm.1   nginx:latest   student-VMware-Virtual-Platform   Running         Running 17 seconds ago

145lcrdqvetr   nginx-swarm.2   nginx:latest   student-VMware-Virtual-Platform   Running         Running 17 seconds ago
```

2. Scale the service to 5 replicas.

3. Verify replicas using:

```
student@student-VMware-Virtual-Platform:~/website$ docker service scale nginx-swarm=5
nginx-swarm scaled to 5
overall progress: 5 out of 5 tasks
1/5: running   [==================================================>]
2/5: running   [==================================================>]
3/5: running   [==================================================>]
4/5: running   [==================================================>]
5/5: running   [==================================================>]
verify: Service nginx-swarm converged
student@student-VMware-Virtual-Platform:~/website$ docker service ls
ID             NAME          MODE         REPLICAS   IMAGE          PORTS
rxy9xb78d1pd   nginx-swarm   replicated   5/5        nginx:latest   *:8888->80/tcp
```

1. Scale the service back to 1 replica.
2. Observe what happens to containers.

```
student@student-VMware-Virtual-Platform:~/website$ docker service scale nginx-swarm=1
nginx-swarm scaled to 1
overall progress: 1 out of 1 tasks
1/1: running   [==================================================>]
verify: Service nginx-swarm converged
student@student-VMware-Virtual-Platform:~/website$ docker service ls
ID             NAME          MODE         REPLICAS   IMAGE          PORTS
rxy9xb78d1pd   nginx-swarm   replicated   1/1        nginx:latest   *:8888->80/tcp
```

Q4. Docker Swarm: Rolling Update

Perform a rolling update in Docker Swarm.

Requirements:

1. Deploy a service using image nginx:1.25 .

```
student@student-VMware-Virtual-Platform:~/website$ docker service create --name nginx-service --replicas 5 -p 8880:80 ng
inx:1.25
sj1raop9ml4yg6h9e5v7ah3nq
overall progress: 5 out of 5 tasks
1/5: running   [==================================================>]
2/5: running   [==================================================>]
3/5: running   [==================================================>]
4/5: running   [==================================================>]
5/5: running   [==================================================>]
verify: Service sj1raop9ml4yg6h9e5v7ah3nq converged
```

2. Update the service to use image nginx:latest .

3. Use rolling update options:

    Update one container at a time

    Delay between updates: 10 seconds

```
student@student-VMware-Virtual-Platform:~/website$ docker service update --image nginx:latest --update-delay 10s nginx-s
ervice
nginx-service
overall progress: 5 out of 5 tasks
1/5: running   [==================================================>]
2/5: running   [==================================================>]
3/5: running   [==================================================>]
4/5: running   [==================================================>]
5/5: running   [==================================================>]
verify: Service nginx-service converged
```

4. Verify update status.

```
student@student-VMware-Virtual-Platform:~/website$ docker service ps nginx-service
ID             NAME                IMAGE          NODE                              DESIRED STATE   CURRENT STATE
           ERROR     PORTS
hxpj32rm9pea   nginx-service.1      nginx:latest   student-VMware-Virtual-Platform   Running         Running about a mi
nute ago
146ishpz2lfw    \_ nginx-service.1  nginx:1.25     student-VMware-Virtual-Platform   Shutdown        Shutdown about a m
inute ago
4f6ajkosb402   nginx-service.2      nginx:latest   student-VMware-Virtual-Platform   Running         Running 39 seconds
 ago
n7cb6yqiw643    \_ nginx-service.2  nginx:1.25     student-VMware-Virtual-Platform   Shutdown        Shutdown 40 second
s ago
o0tp99e59na2   nginx-service.3      nginx:latest   student-VMware-Virtual-Platform   Running         Running about a mi
nute ago
k1hl5my7x5ax    \_ nginx-service.3  nginx:1.25     student-VMware-Virtual-Platform   Shutdown        Shutdown about a m
inute ago
ec471z993dyf   nginx-service.4      nginx:latest   student-VMware-Virtual-Platform   Running         Running 53 seconds
 ago
lv7q0z8jm5tr    \_ nginx-service.4  nginx:1.25     student-VMware-Virtual-Platform   Shutdown        Shutdown 54 second
s ago
84hhfjc0jxbh   nginx-service.5      nginx:latest   student-VMware-Virtual-Platform   Running         Running 25 seconds
 ago
wm4c0y7z8tj7    \_ nginx-service.5  nginx:1.25     student-VMware-Virtual-Platform   Shutdown        Shutdown 26 second
s ago
```

5. Roll back the service if needed.

```
student@student-VMware-Virtual-Platform:~/website$ docker service update --rollback nginx-service
nginx-service
rollback: manually requested rollback
overall progress: rolling back update: 5 out of 5 tasks
1/5: running   [==================================================>]
2/5: running   [==================================================>]
3/5: running   [==================================================>]
4/5: running   [==================================================>]
5/5: running   [==================================================>]
verify: Service nginx-service converged
```

Q5. Docker Swarm: Remove Service and Leave Swarm

Clean up Docker Swarm resources.

Requirements:

1. List all running services.
2. Remove nginx-service .
3. Remove python-service .

```
student@student-VMware-Virtual-Platform:~/website$ docker service ls
ID             NAME             MODE         REPLICAS   IMAGE          PORTS
sj1raop9ml4y   nginx-service    replicated   5/5        nginx:1.25     *:8888->80/tcp
l2cgpteajt5l   python-service   replicated   3/3        my-app:1.0
student@student-VMware-Virtual-Platform:~/website$ docker service rm nginx-service
nginx-service
student@student-VMware-Virtual-Platform:~/website$ docker service rm python-service
python-service
```

4. Verify services are deleted.
5. Leave Docker Swarm mode.

```
student@student-VMware-Virtual-Platform:~/website$ docker service ls
ID        NAME      MODE      REPLICAS   IMAGE     PORTS
student@student-VMware-Virtual-Platform:~/website$ docker swarm leave --force
Node left the swarm.
```

Q6. Explain the usage of Docker Swarm.

1. Container Orchestration: Swarm automates the lifecycle of containers. It manages which physical or virtual machines run specific containers, monitors their health, and automatically replaces dead containers to maintain the application's target state.

2. Service Deployment: Instead of running isolated containers, Swarm uses an abstraction called a

Service. You define the desired configuration (image, ports, volumes), and Swarm takes care of provisioning the containers across the cluster automatically.

3. Scaling Containers: Scaling workloads is instantaneous. By modifying a single parameter via the CLI or a Compose file, you can upscale or downscale the number of active replicas of a service to handle fluctuating application traffic.

4. Load Balancing: Swarm includes an integrated Ingress Routing Mesh. When a request hits any node in the Swarm on an exposed port, the internal load balancer automatically routes that traffic across all running replicas of the service.

5. Rolling Updates: Updates can be rolled out progressively to minimize application downtime. Swarm updates replicas incrementally (e.g., one by one) and introduces delays between updates, allowing the system to verify container health before proceeding.

6. High Availability: Using the Raft consensus protocol, Swarm ensures cluster resilience. If a manager node goes offline, a new leader is elected. If a worker node crashes, Swarm immediately detects the failure and reschedules its running tasks onto healthy nodes.

7. Multi-Node Container Management: Swarm abstracts an entire data centre or cloud infrastructure into a unified cluster. A developer can interact solely with a Manager node, and Swarm will orchestrate workloads seamlessly across dozens of connected Worker nodes.

Q7. Explain the limitations of Docker Swarm.

1. Less Powerful than Kubernetes: Swarm lacks advanced orchestration capabilities. It does not natively provide sophisticated features like complex scheduling constraints, automated self-healing custom logic, automated vertical autoscaling, or fine-grained resource limits.

2. Smaller Ecosystem: Kubernetes has become the undisputed industry standard for container orchestration, causing Swarm’s community footprint to shrink. Consequently, there are far fewer third-party tools, plugins, and guides available.

3. Limited Auto-Scaling Support: Unlike Kubernetes, which features native Horizontal Pod Autoscaling (HPA) based on CPU/Memory metrics, Docker Swarm has no built-in auto-scaling mechanism. Scaling must be performed manually or triggered by custom external shell scripts.

4. Limited Networking Features: While Swarm provides basic overlay networks and routing meshes, it lacks support for cutting-edge networking infrastructure like Service Meshes (e.g., Istio), complex network policies, or native granular ingress controllers.

5. Limited Storage Management: Swarm struggles with stateful, distributed storage management. While it supports standard volume plugins, it does not possess a robust built-in storage orchestration framework equivalent to Kubernetes' Persistent Volumes (PV), Claims (PVC), and dynamic storage provisioners.

6. Not Widely Used in Large Production Systems: Due to its lack of customizability, granular controls,

and multi-tenant isolation, major cloud architectures and massive enterprise deployments overwhelmingly choose managed Kubernetes solutions (like AWS EKS or Google GKE) over Swarm.

7. Fewer Monitoring and Security Integrations: Out-of-the-box monitoring is basic. Security features are rigid, missing advanced multi-layered native integrations such as Role-Based Access Control (RBAC), deep runtime security scanning, and seamless hooks into enterprise monitoring/logging suites.

- If traffic spikes, it automatically updates the desired state from 3 to 10 copies.
- If a container is technically "running" but stops responding to web traffic, Kubernetes flags it as unhealthy, takes it out of the load balancer, and restarts it.